Sprint 11 · Reto «Visión artificial para el cliente»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO **Ficha del reto**

1 Qué os pide el cliente

Qué os pide el cliente. TurisData recibe **cientos de fotos** de alojamientos cada semana y una persona las clasifica a mano (hotel, apartamentos, casa rural, camping, piscina). Quiere saber si un **modelo de visión artificial** (una IA que trabaja con imágenes) puede **pre-clasificarlas** y, sobre todo, **en qué se equivoca**, para decidir cuánto puede fiarse de él y **qué fotos debe revisar una persona**. Lo importante no es el acierto, sino **saber cuándo no fiarse**: un modelo que acierta el 90 % pero no sabe cuándo falla es peligroso; uno que acierta el 85 % y dice «esto no lo tengo claro, que lo revise una persona» es útil. Os pide tres cosas: un **modelo** entrenado con *transfer learning* (reutilizar una red ya entrenada y adaptarla a vuestras clases), un **demostrador** (una función que recibe una foto y responde clase, confianza o «revisar a mano») y un **análisis de errores** de 1–2 páginas.

Cómo se trabaja. No hace falta experiencia previa: el cuaderno `S11_04` trae casi todo el código y cinco ejercicios cortos (el 1 y el 2 los escribís dentro de los pasos 1 y 2 del reto, el día 1 y el día 2; el 3, el 4 y el 5 son la práctica del día 2 con el docente); vuestro trabajo es completarlo, entender los resultados y **mirar las fotos que el modelo falla**. Por defecto se usa un conjunto de **dibujos sintéticos** (no son fotos reales) y, si queréis, uno propio con licencia. En clase tenéis 10 minutos de planning y **25 minutos de reto el día 1**, **80 minutos el día 2** y 30 de entrega, *review* y retro; lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Google Colab (Python con TensorFlow/Keras y scikit-learn ya instalados)	Ejecutar el cuaderno sin instalar nada	En el navegador, con una cuenta de Google. Colab puede desconectarse: ejecutad otra vez desde arriba
El cuaderno del reto `S11_04_reto_vision_cliente`	Es el esqueleto: genera el conjunto de fotos, parte los datos, entrena, evalúa y trae el demostrador	Colab. Haced una copia (*Archivo* → *Guardar una copia en Drive*); su código explicado
Cuadernos de las prácticas: `S11_01` (CNN) y `S11_02` (*transfer learning*)	Las pistas para congelar la red, entrenar la «cabeza» y hacer el ajuste fino	
Presentación y apuntes `S11_02` · apuntes de visión por computador	Repasar qué es una CNN y qué es el *transfer learning*	
Las fotos sintéticas	5 clases (hotel, apartamentos, casa rural, camping, piscina), 60 dibujos por clase; algunos con la etiqueta dudosa a propósito	
Fotos propias (opcional)	Probar con fotos reales: hechas por vosotros o con licencia abierta, **sin personas reconocibles**	
La plantilla del informe `plantilla_analisis_errores.md`	Los seis apartados del informe, con huecos	
Cómo entregar en GitHub	Subir los ficheros a `sprint-11/` y subir el cuaderno de Colab	
Autoevaluación y coevaluación	Cada persona rellena su hoja	
Un asistente de IA (opcional)	Entender errores y pedir ideas	

3 · Guía paso a paso

Son cinco pasos, de **105 minutos en total**: el primero el día 1 (25 minutos) y los otros cuatro el día 2 (80 minutos). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa. Una idea para todo el reto: **primero se mira, después se mide**. Mirad siempre las fotos, no solo los números.

Paso 1 · Preparar los datos (día 1, 25 minutos)

1. Abrir el cuaderno `S11_04` en Colab, hacer una copia y compartirla con el equipo.
2. Decidir si usáis las fotos sintéticas del cuaderno o un conjunto propio.
3. Comprobar cuántas fotos hay por clase y mirarlas.
4. Anotar las etiquetas dudosas y ver cómo se parten los datos.

En sencillo. Antes de entrenar nada hay que saber con qué fotos se trabaja, igual que un cocinero abre la despensa y mira qué ingredientes tiene y en qué estado antes de ponerse a cocinar. En este paso abrís el cuaderno, comprobáis cuántas fotos hay de cada tipo de alojamiento (hotel, apartamentos, casa rural, camping y piscina), **las miráis con vuestros propios ojos** y veis cómo se reparten en tres montones (uno para aprender, otro para ensayar y otro para el examen final). Algunas fotos llevan **a propósito** una etiqueta dudosa, como pasa con los datos reales: vuestro trabajo es encontrarlas. Por defecto no son fotos reales, sino dibujos esquemáticos que genera el propio cuaderno. No hace falta saber programar: escribís una función corta de contar, que tenéis aquí línea a línea.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Celda	Cada caja del cuaderno. Las de código se **ejecutan** con el botón ▶ (o con `Mayús + Intro`).
Función	Una receta con nombre: se escribe una vez con `def` y se usa las veces que haga falta. Aquí escribiréis `contar_por_clase`.
Diccionario	Una lista de parejas «nombre: valor» entre llaves, por ejemplo `{"hotel": 60, "camping": 60}`.
Clase y etiqueta	La clase es el tipo de alojamiento (hotel, camping…). La etiqueta es el nombre de la clase que lleva cada foto; aquí, el de su carpeta.
Dataset sintético	Un conjunto de datos creado por un programa y no recogido de la realidad. Aquí, dibujos de 48 × 48 píxeles, **no fotos reales**.
Etiqueta dudosa	Una foto que está en la carpeta de una clase pero se parece más a otra (por ejemplo, una tienda de campaña en la carpeta de casas rurales).
Partición	El reparto de las fotos en tres montones: **entrenamiento** (con las que aprende el modelo), **validación** (con las que se ensaya y se decide) y **prueba** (el examen final, que se mira una sola vez).
Semilla	Un número que fija el azar para que, al repetir el reparto, salga siempre lo mismo. Aquí, `SEMILLA = 42`.
Dimensiones (`shape`)	El tamaño de una tabla de números. `(180, 48, 48, 3)` son 180 fotos de 48 × 48 píxeles con 3 colores (rojo, verde y azul).

Cómo se hace

1. **Abrir vuestra copia del cuaderno.** Entrad en el cuaderno `S11_04_reto_vision_cliente` (se abre en Colab). Arriba, *Archivo* → *Guardar una copia en Drive*: así trabajáis en una copia vuestra y el original no se toca. Pulsad *Compartir* (arriba a la derecha) y añadid los correos del equipo con permiso de **Editor**. Una persona escribe y las demás miran y opinan; cambiad de turno cada pocos minutos. Colab puede desconectarse: si pasa, volved a ejecutar las celdas desde arriba.
2. **Ejecutar las celdas de preparación, por orden.** Pulsad ▶ en cada una, de arriba abajo: primero «Preparación» (**siempre la primera**: crea la función `dato(…)`), después la de las importaciones y la semilla (`SEMILLA = 42`; tarda unos segundos porque trae TensorFlow, la librería de redes neuronales) y después la del «0. Generador de imágenes sintéticas». Esta última no escribe nada y **no hace falta estudiarla**: solo enseña a Python a dibujar los alojamientos. Sabréis que han terminado porque aparece un y un número entre corchetes. **Ejecutad celda a celda, sin «Ejecutar todo»**: el cuaderno tiene huecos (`...`) que dan error hasta que los rellenéis.

3. **Decidir con qué fotos se trabaja.** Para el nivel mínimo y el estándar vale el conjunto sintético, que es lo que hace la celda del apartado «1 · El dataset: una carpeta por clase». Ejecutadla; su código es este:

```
USAR_DATASET_PROPIO = False
RUTA_PROPIA = "mis_fotos"                       # carpeta con una subcarpeta por clase
RUTA = RUTA_PROPIA if USAR_DATASET_PROPIO else os.path.join("datos_sinteticos", "alojamientos")

if not USAR_DATASET_PROPIO and not os.path.isdir(RUTA):
    crear_dataset_carpetas(RUTA, n_por_clase=60, semilla=7, tam=48, ruido=40, prop_ambiguas=0.12)
    print("Dataset sintético generado en", RUTA)
```

   Qué hace cada línea:

   - `USAR_DATASET_PROPIO = False`: un interruptor. `False` = se usan los dibujos del cuaderno; `True` = se usan vuestras fotos.
   - `RUTA_PROPIA = "mis_fotos"`: el nombre de la carpeta con vuestras fotos, si las usáis. Lo que va tras `#` es un comentario: Python lo ignora.
   - `RUTA = … if … else …`: elige una carpeta u otra según el interruptor. Con las sintéticas, `RUTA` vale `datos_sinteticos/alojamientos`.
   - `if not USAR_DATASET_PROPIO and not os.path.isdir(RUTA):`: «si vais a usar las sintéticas y la carpeta todavía no existe…».
   - `crear_dataset_carpetas(RUTA, n_por_clase=60, …)`: …las crea: 60 dibujos por clase, con `ruido=40` (grano, para que no sean perfectos) y `prop_ambiguas=0.12` (un 12 %, aproximadamente, lleva una etiqueta dudosa a propósito). Solo lo hace la primera vez.
   - `print("Dataset sintético generado en", RUTA)`: avisa de que ya están.

   Si queréis usar **fotos propias** (es un extra), poned `USAR_DATASET_PROPIO = True` y consultad con el docente cómo subirlas a Colab. Reglas: **una carpeta por clase**, con nombres sin espacios ni tildes (`casa_rural`); mínimo 30 fotos por clase (mejor 100 o más); hechas por vosotros o con licencia abierta; **sin personas reconocibles**; y apuntad autor y licencia de cada una.

4. **Ejercicio 1: contar las fotos por clase.** Bajad al enunciado «Ejercicio 1 (comprobar el dataset)». En su celda hay una línea `# TODO` (una pista), tres puntos `...` y varias líneas con `assert`. Borrad **solo los tres puntos** y escribid esto, respetando mayúsculas, comillas, paréntesis y la sangría de 4 espacios por nivel:

```
def contar_por_clase(ruta):
    conteo = {}
    for clase in sorted(os.listdir(ruta)):
        carpeta = os.path.join(ruta, clase)
        if os.path.isdir(carpeta):
            conteo[clase] = sum(f.lower().endswith((".png", ".jpg", ".jpeg")) for f in os.listdir(carpeta))
    return conteo
```

   Qué hace cada línea:

   - `def contar_por_clase(ruta):`: define la función. Recibe `ruta`, el nombre de la carpeta que contiene una subcarpeta por clase. Los dos puntos del final son obligatorios.
   - `conteo = {}`: un diccionario vacío donde se irá apuntando «clase: número de fotos».
   - `for clase in sorted(os.listdir(ruta)):` `os.listdir(ruta)` lista lo que hay dentro de la carpeta; `sorted` lo ordena alfabéticamente; y el `for` repite lo de debajo una vez por cada elemento, llamándolo `clase`.
   - `carpeta = os.path.join(ruta, clase)`: pega la ruta y el nombre para tener el camino completo a esa subcarpeta (por ejemplo, `datos_sinteticos/alojamientos/hotel`).
   - `if os.path.isdir(carpeta):`: «solo si es una carpeta». Es importante: dentro de `RUTA` hay también un archivo suelto, `_ambiguas.csv`, que **no** es una clase.
   - `conteo[clase] = sum(… for f in os.listdir(carpeta))`: para cada archivo `f` de la carpeta pregunta si su nombre acaba en `.png`, `.jpg` o `.jpeg` (`f.lower()` lo pasa a minúsculas para aceptar también `.PNG`; `.endswith((…))` responde `True` o `False`) y `sum` suma los `True` (cada uno vale 1). El resultado se apunta en el diccionario.
   - `return conteo`: devuelve el diccionario terminado. **Sin esta línea la función no devuelve nada.**

   Pulsad ▶. Debajo, el cuaderno imprime el diccionario y comprueba que hay al menos 2 clases, que ninguna está vacía y que, con las sintéticas, hay 60 fotos en cada una de las 5 clases.

5. **Mirar las fotos.** «Primero se mira, después se mide.» Para ver unas cuantas de cada clase, añadid una celda nueva (**+ Código**) y ejecutad:

```
fig, ejes = plt.subplots(len(conteo), 8, figsize=(14, 2 * len(conteo)))
for fila, clase in zip(ejes, conteo):
    for eje, nombre in zip(fila, sorted(os.listdir(os.path.join(RUTA, clase)))):
        eje.imshow(Image.open(os.path.join(RUTA, clase, nombre)))
        eje.set_title(f"{clase}/{nombre}", fontsize=6)
        eje.axis("off")
plt.tight_layout(); plt.show()
```

   Qué hace cada línea:

   - `plt.subplots(len(conteo), 8, …)`: prepara una rejilla con una fila por clase (`len(conteo)` es cuántas hay: 5) y 8 columnas.
   - `for fila, clase in zip(ejes, conteo):`: recorre a la vez las filas de la rejilla y los nombres de las clases.
   - `for eje, nombre in zip(fila, sorted(os.listdir(…))):`: dentro de cada fila, empareja cada hueco con un archivo de esa carpeta (solo caben 8, así que se ven los 8 primeros).
   - `eje.imshow(Image.open(…))`: abre la imagen y la dibuja en su hueco.
   - `eje.set_title(f"{clase}/{nombre}", …)` y `eje.axis("off")`: ponen encima la carpeta y el nombre del archivo, y quitan los ejes numerados.

   Con el conjunto sintético veréis, de forma aproximada: **hotel**, un edificio alto y estrecho con ventanas en filas; **apartamentos**, uno bajo y ancho; **casa_rural**, una casita con tejado rojo; **camping**, una tienda de campaña triangular; **piscina**, un rectángulo azul rodeado de arena. Mirad con calma. **Una foto es dudosa** si su dibujo corresponde a otra clase: por ejemplo, una tienda de campaña en `casa_rural` (en la generación que hemos comprobado, `casa_rural/img_001.png` lo es; si vuestra copia genera otras, buscad las vuestras), un edificio bajo y ancho en `hotel` o uno alto en `apartamentos`. Apuntad las que encontréis en el documento del equipo (os servirá luego), con este formato: `carpeta/nombre → qué dibuja de verdad`. Si queréis ver una foto más grande, abrid el panel de archivos () y haced doble clic sobre ella.

6. **Ver cómo se parten los datos.** Ejecutad la celda «Cargar con `image_dataset_from_directory`». Ya está escrita y **no hay que modificarla**. Sus líneas más importantes:

```
TAM = 48
ds_ent, ds_resto = keras.utils.image_dataset_from_directory(
    RUTA, validation_split=0.4, subset="both", seed=SEMILLA, image_size=(TAM, TAM), batch_size=32, label_mode="int")
NOMBRES = ds_ent.class_names
N_CLASES = len(NOMBRES)
rutas_resto = ds_resto.file_paths
```

```
X_ent, y_ent = a_numpy(ds_ent)
X_resto, y_resto = a_numpy(ds_resto)
mitad = len(X_resto) // 2
X_val, y_val = X_resto[:mitad], y_resto[:mitad]
X_pru, y_pru = X_resto[mitad:], y_resto[mitad:]
rutas_pru = rutas_resto[mitad:]
```

   Qué hace cada parte:

   - `image_dataset_from_directory(RUTA, …)`: carga las fotos de las carpetas y usa **el nombre de cada carpeta como etiqueta**. `validation_split=0.4, subset="both"` devuelve dos montones: el 60 % para entrenamiento (`ds_ent`) y el 40 % restante (`ds_resto`). `seed=SEMILLA` fija el reparto y `image_size=(TAM, TAM)` pone todas las fotos al mismo tamaño.
   - `NOMBRES = ds_ent.class_names`: los nombres de las clases, en orden alfabético. `N_CLASES` es cuántas hay.
   - `mitad = len(X_resto) // 2` y las dos líneas siguientes: el 40 % se parte por la mitad en **validación** (20 %) y **prueba** (20 %). Las dos primeras líneas del segundo bloque convierten las fotos en tablas de números (`X`, las fotos; `y`, su clase).
   - `rutas_pru`: los nombres de archivo de las fotos de prueba, para poder localizarlas luego.

   La prueba no se toca hasta el paso 3: se usa **una sola vez**, al final.

Qué deberíais ver.

- **Al generar las fotos:** `Dataset sintético generado en datos_sinteticos/alojamientos` (solo la primera vez).
- **El ejercicio 1:** `{'apartamentos': 60, 'camping': 60, 'casa_rural': 60, 'hotel': 60, 'piscina': 60}`: las cinco clases, con 60 fotos cada una.
- **La rejilla de fotos:** cinco filas de ocho dibujos, una fila por clase.

- **La partición:** unos mensajes de Keras parecidos a `Found 300 files belonging to 5 classes.`, `Using 180 files for training.` y `Using 120 files for validation.`; después `Clases: ['apartamentos', 'camping', 'casa_rural', 'hotel', 'piscina']` y una línea `entrenamiento (180, 48, 48, 3) · validación (60, 48, 48, 3) · prueba (60, 48, 48, 3)`, más una fila de ocho fotos de ejemplo del entrenamiento, cada una con su clase.

Hecho cuando: `contar_por_clase` pasa sus comprobaciones, habéis mirado las fotos y anotado las dudosas, y todo el equipo sabe que la prueba no se toca hasta el paso 3.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'crear_dataset_carpetas' is not defined` (o `'keras'`, `'os'`, `'CLASES_ALOJ'`)	No se ha ejecutado una celda anterior, o Colab se desconectó y se olvidó de todo.	Ejecutad desde la primera celda hacia abajo, una a una, hasta la que falla.
`NameError: name 'contar_por_clase' is not defined`	La celda del ejercicio 1 todavía tiene los `...` y no define la función.	Borrad los tres puntos, escribid la función y volved a pulsar ▶.
`TypeError: object of type 'NoneType' has no len()`	La función no devuelve nada: falta el `return`.	Poned `return conteo` como última línea, con 4 espacios de sangría.
`NotADirectoryError: [Errno 20] Not a directory: 'datos_sinteticos/alojamientos/_ambiguas.csv'`	Se ha tratado como clase el archivo suelto `_ambiguas.csv`, que crea el cuaderno como lista de apoyo del docente.	Añadid `if os.path.isdir(carpeta):` como en el código de arriba. Ignorad ese archivo: las dudosas se encuentran mirando las fotos.
`AssertionError: Debe haber al menos 2 subcarpetas…`, `Hay alguna clase sin imágenes` o `El dataset sintético debe tener 60 imágenes en cada una de las 5 clases`	El diccionario no sale como se espera.	Escribid `print(conteo)` y mirad qué clase falla. Si la carpeta se quedó a medias, eliminad `datos_sinteticos` en el panel de archivos (, clic derecho → *Eliminar*) y ejecutad de nuevo la celda del apartado 1.
`ValueError: No images found in directory …`	Con fotos propias: la ruta de `RUTA_PROPIA` no existe o sus carpetas no tienen imágenes.	Comprobad el nombre de la carpeta en el panel y que cada clase tenga fotos `.png`, `.jpg` o `.jpeg`.

Paso 2 · Entrenar con *transfer learning* y una referencia (día 2, 25 minutos)

1. Preparar la base preentrenada y hacer el ejercicio 2 (`modelo_transfer`).
2. Entrenar el modelo con la base congelada y después hacer el ajuste fino del último bloque.
3. Entrenar una referencia «desde cero» con la misma arquitectura.
4. Anotar la exactitud de validación de las tres versiones.

En sencillo. Pensad en una persona que ya sabe distinguir rayas, cuadrículas, círculos y triángulos: si ahora le enseñáis a reconocer hoteles y campings, aprenderá más rápido que alguien que no ha visto nunca un dibujo. Eso es el ***transfer learning***: se parte de una red que ya aprendió a ver formas y texturas (la **base**), se le **congela** lo aprendido para no estropearlo y solo se entrena una parte nueva al final (la **cabeza**), que es la que da la respuesta «hotel, camping…». Después, el **ajuste fino** descongela el último bloque de la base y lo retoca muy despacio. Y para saber si todo ese esfuerzo sirve, se entrena también una **referencia «desde cero»**: la misma red sin ningún aprendizaje previo. Al final tenéis tres notas de validación que comparar. No hace falta saber cómo funciona una red por dentro: copiáis, ejecutáis y leéis las cifras.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Transfer learning	Reutilizar una red que ya aprendió una tarea (aquí, reconocer formas y texturas) para una tarea nueva (reconocer alojamientos).
Base preentrenada	La parte de la red que mira la imagen y detecta formas. Ya viene entrenada con una tarea auxiliar.
Cabeza	Las capas nuevas del final de la red, con una neurona por clase. Es lo único que se entrena al principio.

Palabra	Qué significa, en sencillo
Congelar (`trainable = False`)	Impedir que unas capas cambien al entrenar. Lo aprendido se queda como está.
Ajuste fino (*fine-tuning*)	Descongelar el último bloque de la base y entrenarlo un poco más, con retoques muy pequeños.
Tasa de aprendizaje	El tamaño de cada retoque al entrenar. En el ajuste fino es diez veces menor: `1e-4` se lee 0,0001.
Referencia «desde cero»	La misma red sin preentrenar, entrenada con las mismas fotos, para ver cuánto aporta el *transfer learning*.
Época	Una pasada de la red por todas las fotos de entrenamiento. `epochs=30` son 30 pasadas.
Exactitud de validación	La fracción de las 60 fotos de validación que acierta el modelo (0,883 = 88,3 %). Sirve para ensayar y comparar; la prueba no se toca.

Cómo se hace

1. **Preparar la base preentrenada.** Bajad a la sección 2 («El modelo: *transfer learning*») y ejecutad su celda, que ya está escrita y **no hay que modificarla**. El propio cuaderno avisa de que tarda unos 15 segundos. Su código es este:

```
def construir_base(nombre="base"):
    entrada = keras.layers.Input(shape=(TAM, TAM, 3))
    x = keras.layers.Rescaling(1 / 255)(entrada)
    for i, f in enumerate((16, 32, 64), start=1):
        x = keras.layers.Conv2D(f, 3, padding="same", activation="relu", name=f"bloque{i}_conv")(x)
        x = keras.layers.MaxPooling2D(name=f"bloque{i}_pool")(x)
    x = keras.layers.Flatten(name="aplanar")(x)
    return keras.Model(entrada, x, name=nombre)

def con_cabeza(base, n_clases, dropout=0.3, tasa=1e-3):
    entrada = keras.layers.Input(shape=(TAM, TAM, 3))
    x = keras.layers.Dropout(dropout)(base(entrada))
    salida = keras.layers.Dense(n_clases, activation="softmax", name="cabeza")(x)
    m = keras.Model(entrada, salida)
    m.compile(keras.optimizers.Adam(tasa), "sparse_categorical_crossentropy", metrics=["accuracy"])
    return m

# Tarea auxiliar: formas y texturas (ejecuta unos 15 s)
X_aux, y_aux = generar_lote(dibujar_forma, CLASES_FORMAS, 400, semilla=3, tam=TAM)
keras.utils.set_random_seed(SEMILLA)
base_pre = construir_base("base_preentrenada")
con_cabeza(base_pre, len(CLASES_FORMAS), dropout=0.0).fit(X_aux, y_aux, epochs=12, batch_size=64, verbose=0)
print("Base preentrenada lista:", base_pre.count_params(), "parámetros")
```

   Qué hace cada parte:

   - `def construir_base(nombre="base"):`: una función que monta la base. Recibe un nombre para reconocerla.
   - `keras.layers.Input(shape=(TAM, TAM, 3))` y `Rescaling(1 / 255)`: la entrada son imágenes de `TAM` × `TAM` píxeles (48 × 48) con 3 colores, y el valor de cada píxel se pasa de 0–255 a 0–1.
   - `for i, f in enumerate((16, 32, 64), start=1):`: repite tres veces, con `i` = 1, 2, 3 y `f` = 16, 32, 64. Cada vuelta añade un **bloque**.
   - `Conv2D(f, 3, …, name=f"bloque{i}_conv")`: una capa que desliza `f` mirillas pequeñas de 3 × 3 sobre la imagen para detectar formas (bordes, esquinas, rayas). Su **nombre** (`bloque1_conv`, `bloque2_conv`, `bloque3_conv`) importa: el ajuste fino buscará el bloque 3 por el nombre.
   - `MaxPooling2D(name=f"bloque{i}_pool")`: reduce la imagen a la mitad quedándose con lo más llamativo de cada zona.
   - `Flatten(…)` y `return keras.Model(entrada, x, name=nombre)`: alisa el resultado en una lista de números y empaqueta todo como un modelo: la base.
   - `def con_cabeza(base, n_clases, dropout=0.3, tasa=1e-3):`: pone la cabeza sobre una base.
   - `Dropout(dropout)(base(entrada))`: pasa la imagen por la base y apaga al azar un 30 % de las conexiones al entrenar, para que la red no memorice.
   - `Dense(n_clases, activation="softmax", name="cabeza")`: la cabeza: una neurona por clase, y `softmax` convierte sus salidas en probabilidades que suman 1.
   - `m.compile(keras.optimizers.Adam(tasa), …)`: deja el modelo listo para entrenar: `Adam` es el método que va corrigiendo los pesos con retoques de tamaño `tasa`; la pérdida (`sparse_categorical_crossentropy`) mide cuánto se equivoca; y `metrics=["accuracy"]` pide también el porcentaje de aciertos.
   - `X_aux, y_aux = generar_lote(dibujar_forma, CLASES_FORMAS, 400, …)`: crea dibujos de la **tarea auxiliar**: formas y texturas como rayas, cuadrículas, puntos, círculos y triángulos. No son alojamientos: es la «experiencia previa».
   - `keras.utils.set_random_seed(SEMILLA)`: fija el azar para que la red empiece siempre igual.
   - `base_pre = construir_base("base_preentrenada")`: crea la base.
   - `con_cabeza(base_pre, len(CLASES_FORMAS), dropout=0.0).fit(…, epochs=12, …)`: le pone una cabeza **provisional** (una neurona por forma) y la entrena 12 épocas con esos dibujos; así la base aprende a ver formas. La cabeza se tira después. `verbose=0` hace que no escriba nada mientras entrena.
   - `print("Base preentrenada lista:", base_pre.count_params(), "parámetros")`: escribe cuántos números ajustables tiene la base.

2. **Ejercicio 2: `modelo_transfer`.** Bajad al enunciado «Ejercicio 2». En su celda hay un `# TODO`, tres puntos `...` y tres líneas con `assert`. Borrad **solo los tres puntos** y escribid:

```
def modelo_transfer(base_preentrenada, n_clases):
    copia = keras.models.clone_model(base_preentrenada)
    copia.set_weights(base_preentrenada.get_weights())
    copia.trainable = False
    return con_cabeza(copia, n_clases), copia
```

   Qué hace cada línea:

   - `def modelo_transfer(base_preentrenada, n_clases):` : define la función. Recibe la base y el número de clases (5). No olvidéis los dos puntos.
   - `copia = keras.models.clone_model(base_preentrenada)`: hace una **copia** de la base con la misma estructura, pero con pesos nuevos al azar.
   - `copia.set_weights(base_preentrenada.get_weights())`: le pasa a la copia lo que sabe la original (`get_weights()` saca los pesos aprendidos; `set_weights(…)` los pone). Sin esta línea la copia no sabría nada.
   - `copia.trainable = False`: **congela** la copia. Se congela la copia y no la original: el `assert` lo comprueba.
   - `return con_cabeza(copia, n_clases), copia`: devuelve **dos cosas** separadas por una coma (una *tupla*): el modelo completo (copia congelada más cabeza nueva) y la propia copia, que se usará luego en el ajuste fino. La celda siguiente espera exactamente eso, aunque el enunciado no lo diga.

   Pulsad ▶. Si no sale nada, los tres `assert` han pasado: la copia está congelada, la original no, y la salida tiene una neurona por clase.

3. **Entrenar: base congelada, ajuste fino y referencia.** Ejecutad la celda siguiente, que ya está escrita:

```
keras.utils.set_random_seed(SEMILLA)
modelo, base_congelada = modelo_transfer(base_pre, N_CLASES)
modelo.fit(X_ent, y_ent, validation_data=(X_val, y_val), epochs=30, batch_size=16, verbose=0)
print("Exactitud de validación con base congelada:", round(modelo.evaluate(X_val, y_val, verbose=0)[1], 3))

# Fine-tuning del último bloque con tasa pequeña (y recompilando)
base_congelada.trainable = True      # primero se descongela la base entera...
for capa in base_congelada.layers:
    capa.trainable = capa.name.startswith("bloque3")   # ...y solo el último bloque queda entrenable
modelo.compile(keras.optimizers.Adam(1e-4), "sparse_categorical_crossentropy", metrics=["accuracy"])
h = modelo.fit(X_ent, y_ent, validation_data=(X_val, y_val), epochs=25, batch_size=16, verbose=0)
print("Exactitud de validación tras fine-tuning:", round(modelo.evaluate(X_val, y_val, verbose=0)[1], 3))

# Referencia: la misma arquitectura entrenada desde cero
keras.utils.set_random_seed(SEMILLA)
desde_cero = con_cabeza(construir_base("desde_cero"), N_CLASES)
desde_cero.fit(X_ent, y_ent, validation_data=(X_val, y_val), epochs=40, batch_size=16, verbose=0)
print("Exactitud de validación desde cero:", round(desde_cero.evaluate(X_val, y_val, verbose=0)[1], 3))
```

   Qué hace cada parte:

   - `modelo, base_congelada = modelo_transfer(base_pre, N_CLASES)`: usa vuestra función y recoge las dos cosas que devuelve.
   - `modelo.fit(X_ent, y_ent, validation_data=(X_val, y_val), epochs=30, batch_size=16, …)`: entrena 30 épocas con las fotos de entrenamiento, en lotes de 16, y mira cómo va con las de validación. Solo se ajusta la cabeza.
   - `print("Exactitud de validación con base congelada:", round(modelo.evaluate(X_val, y_val, verbose=0)[1], 3))`: `evaluate` devuelve dos cifras (la pérdida y la exactitud); el `[1]` se queda con la exactitud y `round(…, 3)` la deja con tres decimales.
   - `base_congelada.trainable = True`: **descongela** la copia entera, para poder decidir capa a capa. Enseguida se vuelve a congelar casi todo.
   - `for capa in base_congelada.layers:` y `capa.trainable = capa.name.startswith("bloque3")`: recorre las capas de la copia y deja sueltas solo las que se llaman `bloque3…` (`bloque3_conv` y `bloque3_pool`); las demás se quedan congeladas.
   - `modelo.compile(keras.optimizers.Adam(1e-4), …)`: **vuelve a compilar** para que el cambio de congelado surta efecto, con una tasa de aprendizaje diez veces menor (retoques pequeños).
   - `h = modelo.fit(…, epochs=25, …)` y el segundo `print`: 25 épocas más con el último bloque suelto, y la nueva exactitud de validación.
   - `desde_cero = con_cabeza(construir_base("desde_cero"), N_CLASES)`: la **referencia**: la misma arquitectura, con una base nueva que no ha aprendido nada. Se entrena 40 épocas con todo suelto, y el último `print` escribe su exactitud de validación.

   La celda no escribe nada mientras entrena y puede tardar un rato (hasta algún minuto): esperad a que aparezcan las tres líneas.

4. **Anotar las tres cifras.** La celda escribe las tres exactitudes de validación (base congelada, tras el ajuste fino y desde cero). Apuntad en el documento del equipo las tres cifras:

Versión	Exactitud de validación
Transfer learning con la base congelada	(vuestra cifra)
Transfer learning tras el ajuste fino	(vuestra cifra)
Desde cero	(vuestra cifra)

   Solo se mide en validación. La prueba no se toca todavía.

5. **Extra (opcional, necesita internet).** Con fotos reales en Colab podéis probar la rama de MobileNetV2 de la sección 7 del cuaderno (`USAR_INTERNET = True`, un `TAM` de 96 a 160 y `construir_mobilenet(N_CLASES, pesos="imagenet")`). El propio cuaderno avisa de que la descarga de pesos no está verificada en el taller. Si hay dudas, preguntad al docente. Pista de apoyo: el cuaderno de práctica `S11_02` explica el *transfer learning* paso a paso.

Qué deberíais ver.

- **De la celda de la base:** `Base preentrenada lista: 23584 parámetros`. Ese número depende solo de la arquitectura, así que debe salir igual.
- **Del ejercicio 2:** nada, si los `assert` pasan.

- **Del entrenamiento:** dos líneas, `Exactitud de validación con base congelada: 0.…` y `Exactitud de validación tras fine-tuning: 0.…`, y después, de la celda de la referencia, `Exactitud de validación desde cero: 0.…`. Las cifras van de 0 a 1 y, como la validación tiene 60 fotos, cada foto de más o de menos mueve la cifra unos 0,017.
- **Lo esperable:** que el ajuste fino quede igual o mejor que la base congelada, y que el *transfer learning* quede igual o mejor que la referencia. Si no es así, no es un error: anotadlo y pensad por qué (las fotos con ruido, las etiquetas dudosas o que la tarea auxiliar se parezca poco).

Hecho cuando: `modelo_transfer` pasa sus comprobaciones y tenéis anotadas las tres cifras de validación.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'modelo_transfer' is not defined`	La celda del ejercicio 2 todavía tiene los `...` o no se ha ejecutado.	Escribid la función y pulsad ▶ antes de la celda de entrenar.
`NameError: name 'base_pre' is not defined` (o `'X_ent'`, `'N_CLASES'`)	No se ejecutó la celda de la base o las del paso 1, o Colab se desconectó.	Ejecutad desde arriba, por orden, hasta la celda que falla.
`AssertionError: La copia de la base debe estar congelada`	Falta la línea que congela la copia.	Añadid `copia.trainable = False` antes del `return`.
`AssertionError: No congeles la base original: trabaja sobre una copia`	Se ha congelado `base_preentrenada` en vez de `copia`.	Cambiad el nombre: la línea es `copia.trainable = False`.
`AssertionError: La salida debe tener una neurona por clase`	La cabeza no se ha creado con `n_clases`.	Devolved `con_cabeza(copia, n_clases)`, con `n_clases` y no un número escrito a mano.
`TypeError: cannot unpack non-iterable Functional object` (o `NoneType object`)	La función devuelve solo el modelo (o nada) y se esperan dos cosas.	La última línea debe ser `return con_cabeza(copia, n_clases), copia`, con la coma.
`NameError: name 'desde_cero' is not defined`	Se ejecutó la celda de la referencia antes que la de entrenar, o no se ejecutó.	Ejecutad la celda de entrenar entera y esperad a que termine.

Paso 3 · Evaluación y análisis de errores (día 2, 20 minutos)

1. Evaluar en la prueba (**una sola vez**) y sacar las métricas por clase.
2. Mirar la matriz de confusión y hacer los ejercicios 3 y 4.
3. Elegir las fotos mal clasificadas (3 como mínimo, 6 como estándar) y rellenar la tabla de errores.
4. Buscar patrones entre los errores.

En sencillo. Hasta ahora solo habéis mirado la nota de la **validación**, el ensayo. Ahora llega el **examen final**: las 60 fotos de la prueba que el modelo no ha visto nunca. Pero una sola cifra («acierta el 90 %») es como la nota media de una clase: no dice **quién** suspende. Por eso miramos tres cosas: la nota de cada tipo de alojamiento, la **matriz de confusión** (una tabla que cruza «lo que era» con «lo que dijo el modelo») y, sobre todo, **las fotos concretas en las que se equivoca**, con los ojos. Los fallos más peligrosos son aquellos en los que el modelo estaba muy seguro y se equivocó. A veces, además, el que se equivoca no es el modelo, sino la etiqueta de la foto: por eso apuntasteis las dudosas en el paso 1. No hace falta saber matemáticas: dos divisiones y una ordenación.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Prueba (*test*)	Las 60 fotos reservadas desde el paso 1. Se usan **una sola vez**, para esta nota final; después no se decide nada con ellas.
Exactitud global (*accuracy*)	De todas las fotos de la prueba, qué fracción acierta el modelo. El cuaderno la llama «Exactitud en prueba».
Precisión de una clase	De las fotos que el modelo dijo «hotel», cuántas eran de verdad hotel. Baja si el modelo llama «hotel» a cosas que no lo son.
Recall (o exhaustividad)	De las fotos que de verdad eran hotel, cuántas encontró el modelo. Baja si deja hoteles sin reconocer.
F1 macro	Una nota que mezcla precisión y *recall* de cada clase y hace la media de las clases, de modo que todas pesan igual.
Matriz de confusión	Una tabla donde cada **fila** es la clase **real** y cada **columna** la clase **predicha**. La diagonal son los aciertos; lo que queda fuera son los errores.

Palabra	Qué significa, en sencillo
Confianza	La probabilidad más alta que da el modelo a una foto. Un error con confianza alta es un error «seguro de sí mismo».
Hipótesis de causa	Una explicación de por qué falló que se **puede ver en la foto** («la piscina es muy pequeña»), no una frase vaga como «faltan datos».
Índice	La posición de una foto en la lista (la primera es la 0). Con los índices se sacan las fotos concretas.

Cómo se hace

1. **Evaluar en la prueba.** Ejecutad la celda de la sección 3 («Rendimiento en el conjunto de prueba»). Ya está escrita y **no hay que modificarla**. Estas son sus primeras líneas:

```
prob_pru = modelo.predict(X_pru, verbose=0)
pred_pru = prob_pru.argmax(axis=1)
pred_cero = desde_cero.predict(X_pru, verbose=0).argmax(axis=1)
print(f"Exactitud en prueba → transfer + fine-tuning: {accuracy_score(y_pru, pred_pru):.3f} · desde cero:
{accuracy_score(y_pru, pred_cero):.3f}")
print(f"F1 macro → transfer: {f1_score(y_pru, pred_pru, average='macro'):.3f} · desde cero: {f1_score(y_pru, pred_cero,
average='macro'):.3f}")
print(classification_report(y_pru, pred_pru, target_names=NOMBRES, zero_division=0))

cm = confusion_matrix(y_pru, pred_pru, labels=range(N_CLASES))
```

   Qué hace cada línea:

   - `prob_pru = modelo.predict(X_pru, verbose=0)`: el modelo mira cada foto de la prueba y devuelve 5 probabilidades por foto, una por clase. Es una tabla de 60 filas y 5 columnas.
   - `pred_pru = prob_pru.argmax(axis=1)`: de cada fila, la **posición** de la probabilidad más alta. Es la clase que el modelo elige (un número del 0 al 4).
   - `pred_cero = desde_cero.predict(…).argmax(axis=1)`: lo mismo con el modelo de referencia, para compararlos.
   - Los dos `print(f"…")`: escriben la exactitud global y el F1 macro de los dos modelos, con tres decimales. `y_pru` son las clases reales; `pred_pru`, las predichas.
   - `print(classification_report(…))`: una tabla con una fila por clase y las columnas `precision`, `recall`, `f1-score` y `support` (cuántas fotos de prueba hay de esa clase). Debajo, las filas `accuracy`, `macro avg` y `weighted avg`.
   - `cm = confusion_matrix(…)`: la matriz de confusión, de 5 × 5. Cada fila es una clase real; cada columna, una predicha.

   El resto de la celda dibuja la matriz como cuadrados azules con el número dentro (más oscuro = más fotos). No hace falta tocarlo. **Esta es la única celda que evalúa en la prueba: no la ejecutéis otra vez para retocar el modelo.**

2. **Ejercicio 3: precisión y *recall* de cada clase.** Bajad al enunciado «Ejercicio 3». En su celda hay un `# TODO`, tres puntos `...` y varias líneas con `assert`. Borrad **solo los tres puntos** y escribid:

```
diag = np.diag(cm).astype(float)
recall_clase = np.divide(diag, cm.sum(axis=1), out=np.zeros(N_CLASES), where=cm.sum(axis=1) > 0)
precision_clase = np.divide(diag, cm.sum(axis=0), out=np.zeros(N_CLASES), where=cm.sum(axis=0) > 0)
clase_mas_dificil = NOMBRES[int(np.argmin(recall_clase))]
```

   Qué hace cada línea:

   - `diag = np.diag(cm).astype(float)`: saca la diagonal de la matriz (los aciertos de cada clase, uno por clase) y los pasa a números con decimales.
   - `cm.sum(axis=1)`: la suma de cada **fila**, es decir, cuántas fotos reales hay de cada clase. `cm.sum(axis=0)`: la suma de cada **columna**, es decir, cuántas fotos ha llamado el modelo de cada clase.
   - `recall_clase = np.divide(diag, cm.sum(axis=1), …)`: aciertos entre fotos reales de la clase. Es el *recall*.
   - `precision_clase = np.divide(diag, cm.sum(axis=0), …)`: aciertos entre fotos que el modelo llamó de esa clase. Es la precisión de la clase.
   - `out=np.zeros(N_CLASES), where=… > 0`: es la forma de «si una suma es 0, deja 0» que pide el enunciado. `np.divide` solo divide donde la suma es mayor que 0; en el resto deja el 0 de `out`, y así no hay divisiones entre cero.
   - `clase_mas_dificil = NOMBRES[int(np.argmin(recall_clase))]`: `np.argmin` da la posición del *recall* más bajo, y `NOMBRES[…]` la convierte en el **nombre** de la clase (hay que guardar el nombre, no el número).

   Pulsad ▶. La celda imprime una tabla con una fila por clase y las columnas `recall` y `precisión`, y la línea `Clase más difícil (menor recall): …`. Las cifras de la tabla deben **coincidir** (redondeadas) con las columnas `recall` y `precision` del informe de arriba: es una buena forma de comprobar que lo habéis hecho bien.

3. **Ejercicio 4: ordenar los errores de más a menos confianza.** Bajad al enunciado «Ejercicio 4», borrad los `...` y escribid:

```
def indices_errores(y_real, prob):
    pred = prob.argmax(axis=1)
    errores = np.where(pred != y_real)[0]
    return errores[np.argsort(-prob.max(axis=1)[errores])]
```

   Qué hace cada línea:

   - `def indices_errores(y_real, prob):`: define la función. Recibe las clases reales y la tabla de probabilidades. No olvidéis los dos puntos.
   - `pred = prob.argmax(axis=1)`: la clase elegida para cada foto, como antes.
   - `errores = np.where(pred != y_real)[0]`: `pred != y_real` marca con `True` las fotos donde lo predicho no coincide con lo real; `np.where` da las **posiciones** de esos `True`. Son los índices de las fotos mal clasificadas.
   - `prob.max(axis=1)[errores]`: la confianza (la probabilidad más alta) de cada foto mal clasificada.
   - `np.argsort(-…)`: da el orden que pondría las confianzas de **mayor a menor**. El signo menos delante es lo que invierte el orden (sin él saldrían de menor a mayor).
   - `return errores[…]`: devuelve los índices de los errores en ese orden: primero los más seguros de sí mismos.

   Pulsad ▶. Las comprobaciones de debajo usan números de juguete (por eso no hace falta haber entrenado nada para probarlas).

4. **Ver los errores.** Ejecutad las dos celdas siguientes (ya escritas). La primera escribe `N imágenes mal clasificadas de 60` y dibuja hasta **10** fotos: encima de cada una pone la clase real (`real:`), la predicha (`predicho:`) y entre paréntesis la confianza, y junto a ella, en letra pequeña, el **nombre del archivo**. Las primeras son las de más confianza, las más preocupantes. La segunda, con `value_counts`, cuenta los **pares** más repetidos con el formato `clase real → clase predicha`.

   Para cada foto de la figura mirad con calma y preguntaos: ¿está mal etiquetada y el modelo tiene razón? ¿Tiene mucho grano? ¿Se parece a otra clase (un edificio alto y uno bajo, por ejemplo)? ¿El objeto es muy pequeño? Si la foto aparece en vuestra lista de «etiquetas dudosas» del paso 1, probablemente **no** se ha equivocado el modelo, sino quien etiquetó, y eso también hay que contarlo.

5. **Rellenar la tabla de errores.** Copiad esta tabla en `analisis_errores.md` (lo escribiréis en el paso 5) y añadid **una fila por foto**: 3 como mínimo, 6 para el nivel estándar. *(Si el modelo acierta casi todo y hay menos errores de los que se piden, completad la tabla con las fotos que acertó con menos confianza: es un «casi error», se analiza igual y lo ponéis en el informe. Si no hay ningún error, decidlo y analizad las 3 de menos confianza.)* En «Foto» poned la carpeta de la clase real y el nombre del archivo, por ejemplo `camping/img_031.png`:

Foto	Verdadera	Predicha	Confianza	Hipótesis de causa	Mejora propuesta
`piscina/img_012.png` (ejemplo inventado)	Piscina	Hotel	81 %	El edificio ocupa media imagen y la piscina es muy pequeña	Más ejemplos con la piscina en primer plano

   Una hipótesis vale cuando **se ve en la foto**: «por falta de datos» no basta. Mejoras posibles: más fotos de esa clase, revisar la etiqueta, añadir variaciones de las fotos que ya hay (girarlas, recortarlas), o más ajuste fino. La fila de ejemplo es inventada: no la copiéis.

6. **Interpretar la matriz en una frase.** Mirad qué números hay fuera de la diagonal y escribid, con vuestras cifras: *«La clase [X] es la más difícil (*recall *[..]): [n] de sus fotos se predicen como [Y], posiblemente porque [se parecen / la etiqueta es dudosa]»*. Comprobad también si el modelo con *transfer learning* y el de «desde cero» quedan cerca o lejos en las dos líneas de precisión y F1 macro, y anotadlo.

Qué deberíais ver.

- **Dos líneas** con la exactitud y el F1 macro de los dos modelos, y el informe `classification_report` con cinco filas de clases (`apartamentos`, `camping`, `casa_rural`, `hotel`, `piscina`) más las filas `accuracy`, `macro avg` y `weighted avg`.
- **La matriz de confusión:** una cuadrícula de 5 × 5 con números, con la diagonal más oscura. Los dibujos de hotel y apartamentos se parecen (los dos son edificios), así que sería razonable que se mezclen entre sí, pero lo que cuenta es lo que salga en **vuestra** matriz.
- **El ejercicio 3:** una tabla de cinco filas con `recall` y `precisión`, y la clase más difícil.
- **Los errores:** `N imágenes mal clasificadas de 60`, hasta diez fotos con su clase real, predicha y confianza, y la lista de los pares más confundidos. Si N es menor que 3, es una buena noticia, no un fallo: apuntadlo y avisad al docente de que no tenéis fotos suficientes para la tabla.

Hecho cuando: tenéis las métricas por clase y la matriz interpretada en una frase, y la tabla con al menos 3 errores (6 si vais al estándar), cada uno con hipótesis y mejora. **A partir de aquí la prueba no se vuelve a usar para decidir nada.**

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'cm' is not defined` (o `'prob_pru'`, `'pred_pru'`)	No se ha ejecutado la celda de la sección 3 que evalúa.	Ejecutadla (y las anteriores, por orden) y repetid el ejercicio.
`NameError: name 'recall_clase' is not defined` o `name 'indices_errores' is not defined`	La celda del ejercicio todavía tiene los `...` o no se ha ejecutado.	Borrad los tres puntos, escribid el código y pulsad ▶.
`AssertionError: El recall se divide entre la suma de la FILA`	Se ha dividido entre la suma de columnas (`axis=0`) en lugar de la de filas.	El *recall* usa `cm.sum(axis=1)`; la precisión, `cm.sum(axis=0)`.
`AssertionError: clase_mas_dificil debe ser el nombre de una clase`	Se ha guardado un número (el resultado de `np.argmin`) y no el nombre.	Envolvedlo con `NOMBRES[int(np.argmin(recall_clase))]`.
`AssertionError: Deben ser vectores de una posición por clase`	`recall_clase` o `precision_clase` no tiene 5 valores.	Comprobad que usáis `np.diag(cm)` y las sumas con `axis`, no una suma de toda la matriz.
`AssertionError: Los errores son las filas 0 (confianza 0.9) y 2 (0.8); deben salir en ese orden: [0, 2]`	Los errores salen desordenados o de menor a mayor confianza.	Poned el signo menos dentro de `np.argsort(-…)`.
`IndentationError: expected an indented block after function definition on line …`	Las líneas de dentro de `indices_errores` no tienen sangría.	Poned 4 espacios delante de cada línea de la función, incluido el `return`.

Paso 4 · Demostrador y umbral de confianza (día 2, 15 minutos)

1. Probar `clasificar_foto` con 2 o 3 fotos de la prueba.
2. Hacer el ejercicio 5: medir cobertura y acierto para varios umbrales.
3. Elegir el umbral y escribir la recomendación en una frase.

En sencillo. Imaginad que contratáis a alguien para clasificar fotos y le decís: «Si estás seguro, decide tú; si dudas, pásame la foto y la miro yo». Eso es lo que hace un modelo responsable. La **confianza** es lo seguro que está (de 0 a 1) y el **umbral** es la línea que ponéis: por encima de ella, el modelo decide solo; por debajo, la foto va a «revisar a mano». Si ponéis el umbral muy alto, el modelo decide pocas fotos pero casi siempre acierta; si lo ponéis muy bajo, decide casi todas pero se equivoca más. En este paso probáis la función que hace esto con fotos de verdad, medís las dos cosas con una función nueva y elegís dónde poner la línea. Lo importante no es sacar un número «bonito», sino **poder explicarle al cliente cuántas fotos revisa una persona y por qué**.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Demostrador	Una función que recibe **una foto** y responde con la clase y lo segura que está. Es lo que enseñaréis al cliente.
Confianza	La probabilidad más alta que da el modelo a una foto, de 0 (nada seguro) a 1 (seguro del todo).
Umbral	La confianza mínima para que el modelo decida solo. Por debajo de ella, responde «revisar a mano».
Cobertura	De todas las fotos, la fracción que el modelo decide solo (las que superan el umbral).
Exactitud de las aceptadas	De las fotos que el modelo decide solo, la fracción que acierta.
Abstenerse	No decidir y pasar la foto a una persona. Es lo que hace el modelo con las fotos que no supera el umbral.
Tupla	Un par o grupo de valores entre paréntesis, como `(0.5, 1.0)`. Una función puede devolver varios valores a la vez en una tupla.
Máscara	Una lista de `True` y `False`, una por foto, que dice cuáles cumplen una condición. Aquí: cuáles superan el umbral.
Argumento con nombre	Pasar un valor escribiendo su nombre: `umbral=0.9`. Es obligatorio en `clasificar_foto`, como se explica abajo.

Cómo se hace

1. **Probar el demostrador.** Bajad a la sección 6 («Demostrador») y ejecutad su celda. Ya está escrita y **no hay que modificarla**. Su código es este:

```
def clasificar_foto(ruta_imagen, modelo_=None, umbral=0.7):
    """Devuelve (clase, confianza, probabilidades por clase) o 'revisar a mano' si la confianza es baja."""
    modelo_ = modelo_ or modelo
    img = keras.utils.load_img(ruta_imagen, target_size=(TAM, TAM))
    p = modelo_.predict(np.expand_dims(keras.utils.img_to_array(img), 0), verbose=0)[0]
    clase = NOMBRES[int(p.argmax())] if p.max() >= umbral else "revisar a mano"
    return clase, float(p.max()), dict(zip(NOMBRES, p.round(3)))

demo_ruta = rutas_pru[0]
print("Foto:", demo_ruta, "· real:", NOMBRES[y_pru[0]])
print(clasificar_foto(demo_ruta))
```

   Qué hace cada línea:

   - `def clasificar_foto(ruta_imagen, modelo_=None, umbral=0.7):`: recibe la ruta de una foto, opcionalmente un modelo (si no se dice, usa el vuestro) y el umbral (0,7 si no se dice).
   - `modelo_ = modelo_ or modelo`: «si no me han dado modelo, uso `modelo`», el que entrenasteis en el paso 2.
   - `img = keras.utils.load_img(ruta_imagen, target_size=(TAM, TAM))`: abre la foto del disco y la deja del tamaño que espera la red (48 × 48).
   - `p = modelo_.predict(np.expand_dims(keras.utils.img_to_array(img), 0), verbose=0)[0]`: pasa la foto a números, le añade una dimensión (la red espera un lote de fotos, aunque sea de una) y obtiene las 5 probabilidades. El `[0]` se queda con la única fila.
   - `clase = NOMBRES[int(p.argmax())] if p.max() >= umbral else "revisar a mano"`: si la probabilidad más alta llega al umbral, la clase es la de esa probabilidad; si no, la respuesta es `"revisar a mano"`.
   - `return clase, float(p.max()), dict(zip(NOMBRES, p.round(3)))`: devuelve tres cosas: la clase (o «revisar a mano»), la confianza y un diccionario con la probabilidad de cada clase, con tres decimales.
   - `demo_ruta = rutas_pru[0]` y los dos `print`: eligen la primera foto de la prueba, escriben su ruta y la clase real y muestran lo que responde la función.

   La misma celda termina guardando el modelo en el archivo `clasificador_alojamientos.keras`, recargándolo y comprobando que responde igual (`modelo.save(…)`, `keras.models.load_model(…)`). No hay que hacer nada; sirve para asegurar que el modelo se puede guardar y reutilizar. El archivo queda en la carpeta del cuaderno (en Colab, en el panel de archivos de la izquierda, con el icono de la carpeta): si queréis entregarlo, descargadlo desde ahí (botones de los tres puntos → *Descargar*) y subidlo a `sprint-11/`.

2. **Probar con 2 o 3 fotos.** En una celda nueva (**+ Código**) ejecutad:

```
for i in (0, 1, 2):
    print("real:", NOMBRES[y_pru[i]], "→", clasificar_foto(rutas_pru[i], umbral=0.9)[:2])
```

   Qué hace cada línea:

   - `for i in (0, 1, 2):`: repite lo de debajo tres veces, con `i` valiendo 0, 1 y 2 (las tres primeras fotos de la prueba). Cambiad los números para ver otras.
   - `print("real:", NOMBRES[y_pru[i]], "→", …)`: escribe la clase real de la foto `i` y, a continuación, la respuesta del demostrador.
   - `clasificar_foto(rutas_pru[i], umbral=0.9)[:2]`: llama a la función con un umbral de 0,9; el `[:2]` se queda solo con los dos primeros valores de lo que devuelve (la clase y la confianza) para que sea más corto.

   Importante: el umbral se escribe siempre con su nombre (`umbral=0.9`). La función tiene como segundo argumento el modelo (`modelo_`): si escribís `clasificar_foto(ruta, 0.9)`, Python cree que 0,9 es el modelo y da error. Probad también `umbral=0.5` y `umbral=0.99` con la misma foto y fijaos en cómo cambia la respuesta.

3. **Ejercicio 5: cobertura y exactitud de las aceptadas.** Bajad al enunciado de la sección 5 («IA responsable»), borrad los tres puntos `...` de su celda y escribid:

```
def cobertura_y_exactitud(prob, y_real, umbral):
    aceptadas = prob.max(axis=1) >= umbral
    if aceptadas.sum() == 0:
        return 0.0, 0.0
    aciertos = prob.argmax(axis=1)[aceptadas] == y_real[aceptadas]
    return float(aceptadas.mean()), float(aciertos.mean())
```

   Qué hace cada línea:

   - `def cobertura_y_exactitud(prob, y_real, umbral):`: define la función. Recibe la tabla de probabilidades, las clases reales y el umbral. Los dos puntos del final son obligatorios.
   - `aceptadas = prob.max(axis=1) >= umbral`: `prob.max(axis=1)` es la confianza de cada foto. Comparada con el umbral da una **máscara**: `True` para las fotos que el modelo decide solo.
   - `if aceptadas.sum() == 0:`: `sum()` cuenta los `True`. Si no se acepta ninguna foto no se puede calcular nada…
   - `return 0.0, 0.0`: …así que se devuelve `(0.0, 0.0)`, que es lo que pide el enunciado. Esta línea va con **8 espacios** (está dentro del `if`).
   - `aciertos = prob.argmax(axis=1)[aceptadas] == y_real[aceptadas]`: se queda solo con las fotos aceptadas (`[aceptadas]`) y comprueba, para cada una, si la clase elegida coincide con la real. El resultado es otra lista de `True` y `False`.
   - `return float(aceptadas.mean()), float(aciertos.mean())`: la media de una lista de `True`/`False` es la fracción de `True`. La primera es la **cobertura** (fracción de fotos aceptadas); la segunda, la **exactitud de las aceptadas**. Se devuelven juntas, en una tupla, y siempre en este orden.

   Pulsad ▶. Los `assert` de debajo usan una tabla de juguete de 4 fotos.

4. **Medir varios umbrales.** Ejecutad la celda siguiente, que ya está escrita: calcula las probabilidades de la **validación** (`prob_val`) y, con ellas, la cobertura y la «exactitud en las aceptadas» para los umbrales `0.0`, `0.5`, `0.7`, `0.9` y `0.95`. Se usa la validación y no la prueba porque el umbral se **elige** mirando esta tabla, y la prueba tiene que seguir sin tocarse hasta el final. Fijaos en tres cosas: con umbral `0.0` se acepta todo (cobertura 1) y la exactitud es la de la validación (no tiene por qué coincidir con la del paso 3, que es de la prueba); **la cobertura solo puede bajar** al subir el umbral; y la exactitud de las aceptadas suele subir (no siempre). Con solo 60 fotos, una foto más o menos mueve las cifras unos 1,7 puntos: no saquéis conclusiones de diferencias pequeñas.
5. **Elegir el umbral y medirlo en la prueba.**
   - **Estándar:** os basta con el umbral que trae el cuaderno (`0.7`). Ejecutad la celda de debajo de la tabla, que ya está escrita: usa ese umbral con la **prueba** y escribe una línea con la cobertura y el acierto reales.
   - **Extra (el nivel «extra» de la rúbrica):** elegirlo con datos. Mirad la tabla de validación y elegid el umbral que equilibre fotos revisadas a mano y errores evitados. Después, en esa misma celda de debajo, cambiad `umbral_elegido = 0.7` por el vuestro y ejecutadla **una sola vez**: así la prueba solo mide el resultado y no ha servido para decidir nada.

   Pensad también qué error es peor para TurisData: confundir «apartamentos» con «hotel» o clasificar un camping como piscina. Cuanto más cuesta equivocarse, más alto debería ser el umbral.
6. **La frase para el cliente.** Escribidla ya, con los números de la línea de la prueba: *«Con un umbral de […], el modelo decide solo el […] % de las fotos y acierta el […] % de ellas; el resto ([…] %) lo revisa una persona»*. Si la cobertura es 0,8 (el 80 %), el resto, un 20 %, lo revisa una persona: son las fotos para revisar.

Qué deberíais ver.

- **Del demostrador:** una tupla de tres valores: la clase (o `revisar a mano`), la confianza con muchos decimales (por ejemplo `0.8999…`) y un diccionario con las 5 probabilidades. Con NumPy reciente, las probabilidades pueden salir como `np.float32(0.9)` en vez de `0.9`: es el mismo número.
- **Del bucle de 3 fotos:** tres líneas `real: … → ('…', 0.…)`, a veces con `'revisar a mano'`.
- **De la tabla de umbrales (validación):** cinco filas con las columnas `cobertura` y `exactitud en las aceptadas`, con la cobertura bajando de arriba abajo.
- **De la celda de la prueba:** una línea parecida a `Prueba con umbral 0.7: el modelo decide solo el 78% de las fotos y acierta el 77% de esas` (con vuestras cifras).

Hecho cuando: `clasificar_foto` funciona con 2 o 3 fotos delante del equipo, `cobertura_y_exactitud` pasa sus comprobaciones y tenéis anotado el umbral elegido con su frase.

Si algo falla

Veis…	Qué significa	Qué hacer
`AttributeError: 'float' object has no attribute 'predict'`	Habéis escrito el umbral sin su nombre, así que Python lo toma como el modelo.	Escribid `clasificar_foto(ruta, umbral=0.9)`.
`NameError: name 'rutas_pru' is not defined` (o `'modelo'`, `'y_pru'`)	Colab se desconectó o no se ejecutaron las celdas de los pasos 1 a 3.	Ejecutad desde arriba, por orden, hasta la sección 3 y después el demostrador.
`FileNotFoundError: [Errno 2] No such file or directory: …`	La ruta de la foto no existe: está mal escrita o la carpeta `datos_sinteticos` se ha borrado.	Usad `rutas_pru[0]`, `rutas_pru[1]`… en vez de escribir la ruta a mano. Si se ha borrado, volved a ejecutar la celda de la sección 1.
`NameError: name 'cobertura_y_exactitud' is not defined`	La celda del ejercicio 5 todavía tiene los `...`.	Escribid la función y volved a pulsar ▶.
`AssertionError: Con umbral 0.65 se aceptan 2 de 4 (filas 0 y 2) y ambas aciertan`	La cuenta o el orden están mal: se calculan los aciertos sobre todas las fotos y no solo sobre las aceptadas, o se devuelve primero la exactitud y después la cobertura.	Revisad que `aciertos` usa `[aceptadas]` y que se devuelve `(cobertura, exactitud)`, en ese orden.
`AssertionError: Si no se acepta ninguna: (0.0, 0.0)`	Falta el caso especial, o devuelve otra cosa.	Añadid el `if aceptadas.sum() == 0:` con `return 0.0, 0.0` dentro.
`IndentationError: expected an indented block after 'if' statement on line …`	El `return` de dentro del `if` no tiene sangría.	Ponedle 8 espacios (4 del `def` y 4 más del `if`).

Paso 5 · Informe, README y entrega (día 2, 20 minutos)

1. Escribir `analisis_errores.md` con lo que ya habéis ido apuntando.
2. Crear el `README.md`.
3. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado al final del README.
4. Reiniciar y ejecutar todo el cuaderno, bajarlo y subir los ficheros a `sprint-11/`.

En sencillo. Es el paso de **cerrar el trabajo**. Primero, contar a TurisData qué habéis descubierto: en qué se equivoca el modelo, qué fotos puede decidir solo y cuáles tiene que mirar una persona. Ya tenéis casi todo apuntado en los pasos anteriores: ahora se ordena. Segundo, el README, que es la hoja de entrada de la carpeta: sirve para que otra persona pueda repetir vuestro cuaderno sin preguntaros nada. Tercero, alguien que no escribió esa parte lo revisa con la rúbrica y deja su opinión por escrito. Cuarto, subir todo al repositorio del equipo desde el navegador. No hay código nuevo.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Informe de análisis de errores	El documento de 1–2 páginas (`analisis_errores.md`) que cuenta resultados, errores y recomendación al cliente.
Markdown (`.md`)	Texto normal con unas pocas marcas: # para un título, **…** para negrita y barras verticales para las tablas. GitHub lo muestra ya con formato.
README	La hoja de entrada: qué hay, de dónde salen las fotos, cómo se repite y quién hizo qué. Es lo primero que lee el docente.
Riesgo de IA responsable	Algo concreto que podría salir mal al usar el modelo (por ejemplo, que los dibujos no sean fotos reales) junto con una **medida** para evitarlo.
Licencia	El permiso que dice si podéis usar una foto y con qué condiciones. Hay que apuntar autor y licencia de cada foto que no sea vuestra.
Rúbrica	La tabla con la que se evalúa el reto: cuatro criterios, cada uno de 1 a 4.
Commit	Cada vez que guardáis un cambio en GitHub con el botón verde **Commit changes**. Queda apuntado con el nombre de quien lo hace.
Reiniciar y ejecutar todo	Borra la memoria del cuaderno y ejecuta todas las celdas de arriba abajo. Prueba que el cuaderno funciona solo y deja los resultados visibles.

Cómo se hace

1. **Repartir el trabajo.** Cada persona sube lo suyo con su cuenta: así el historial muestra a todo el equipo, que es lo que se evalúa. Un reparto posible: una persona prepara y sube el cuaderno, otra escribe el informe y otra el README. Quien revise con la rúbrica (el punto 4) **no puede ser quien escribió esa parte**.
2. **Escribir el informe** `analisis_errores.md`. Abrid `plantilla_analisis_errores.md`, pulsad *Raw*, copiad el texto y, en la carpeta `sprint-11` del repositorio, usad *Add file → Create new file* con el nombre `analisis_errores.md` y pegadlo. Tiene seis apartados **en este orden**; sustituid lo que va entre corchetes por vuestras cifras y frases.

   Casi todo sale de lo que ya apuntasteis: las fotos y la partición (pasos 1 y 2), las cifras por clase y la matriz (paso 3), la tabla de errores (paso 3) y la frase del umbral (paso 4).

   Para el nivel mínimo bastan los apartados 1, 3 y 4. Los riesgos tienen que ser **de este caso**, no frases generales como «la IA puede tener sesgos». Dos ejemplos del tipo que se pide: *«Riesgo: los dibujos son sintéticos, no fotos reales, así que las cifras pueden no valer con fotos de verdad. Medida: repetir la prueba con fotos reales con licencia antes de usar el modelo»*; *«Riesgo: el modelo clasificó con [..] % de confianza una foto de [..] como [..]. Medida: umbral de [..] y revisión de una persona por debajo»*. Pensad también en las etiquetas dudosas: parte de los errores pueden ser fallos de etiquetado y no del modelo.

   No vale: un informe sin recomendación, cifras escritas de memoria o copiadas de lo que dijo una IA (todas salen de vuestro cuaderno), hipótesis que no se ven en la foto, ni riesgos genéricos.
3. **Rellenar el README.** Ya existe `sprint-11/README.md` en el repositorio del equipo, con la plantilla de cuatro campos. Abridlo (icono del lápiz) y sustituid su contenido por estas secciones; para el nivel mínimo bastan cinco, en pocas líneas. Así puede quedar:

   ```
   # Sprint 11 · Visión artificial para el cliente

   ## Qué es
   Un modelo que clasifica fotos de alojamientos de TurisData en cinco tipos (hotel, apartamentos, casa rural, camping y
   piscina) y dice cuándo hay que revisar la foto a mano. Se entrega el cuaderno `reto_vision_cliente.ipynb` y el informe
   `analisis_errores.md`.

   ## Cómo se ejecuta
   Abrir `reto_vision_cliente.ipynb` en Colab y pulsar *Entorno de ejecución → Reiniciar y ejecutar todo*. La semilla es 42
   (`SEMILLA = 42`).

   ## Datos
   Dibujos sintéticos que genera el propio cuaderno (60 por clase). No son fotos reales. [Si usáis fotos propias: autor y
   licencia, y cuántas hay por clase.]

   ## Uso de IA y límites
   Pedimos a la IA [..]; comprobamos a mano [..]; cambiamos [..]. Límite: los datos son sintéticos y con etiquetas dudosas a
   propósito.

   ## Quién hizo qué
   - Persona 1 (@usuario): cuaderno y ejercicios 1 y 2
   - Persona 2 (@usuario): evaluación y ejercicios 3 y 4
   - Persona 3 (@usuario): umbral, ejercicio 5, informe y README
   ```

   Para el nivel extra, añadid en «Cómo se ejecuta» las versiones de Python y de TensorFlow (`print(tf.__version__)` en una celda del cuaderno, que ya trae `tf` cargado). Después pulsad **Commit changes**.

4. **La revisión con la rúbrica (5 minutos).** Quien revisa **no escribió esa parte**. Sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica: ejecutar el cuaderno, leer el informe y el README como si fuera el cliente, mirar dos errores con la foto delante, poner un nivel de 1 a 4 por criterio **con una frase de prueba** y apuntar el resultado. Añade al final del `README.md` esta sección rellena:

```
## Revisión con la rúbrica

Revisado por: (nombre) · Fecha: (dd/mm/aaaa)

| Criterio | Nivel (1–4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | | |
| Reproducibilidad | | |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

Lo primero que mejoraríamos: (una cosa concreta)
```

   Un ejemplo de frase de prueba: «3 · se ejecuta de arriba abajo y la referencia sale en la misma tabla, pero la prueba se usa dos veces». Sin frase, el nivel no vale. Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos».

5. **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub. Para el cuaderno: en Colab, ponedle el nombre `reto_vision_cliente.ipynb` (*Archivo → Cambiar nombre*), después *Entorno de ejecución → Reiniciar y ejecutar todo*, mirad que ninguna celda dé error y *Archivo → Descargar → Descargar .ipynb*. En el repositorio del equipo: (1) **entrad primero en la carpeta `sprint-11`**; (2) *Add file → Upload files*; (3) arrastrad el fichero; (4) escribid un mensaje corto y pulsad **Commit changes**. Un reparto posible: la persona A sube `reto_vision_cliente.ipynb` («Añado cuaderno con el modelo y el demostrador»), la B sube `analisis_errores.md` («Añado análisis de errores») y la C sube `README.md` («Añado README»). Los nombres de archivo, **en minúsculas y sin espacios**. Las fotos no se suben: el cuaderno vuelve a generarlas.
6. **La autoevaluación.** Cada persona copia la plantilla de autoevaluación y coevaluación, la guarda con su nombre (`autoevaluacion_nombre.md`, sin espacios ni tildes), la rellena y la sube a `sprint-11/`. La coevaluación va dentro del mismo fichero y se hace durante la *review*.

Qué deberíais ver. En la carpeta `sprint-11` del repositorio, los tres ficheros del equipo y una hoja de autoevaluación por persona (**3 + N**); y en el historial (el icono del reloj), al menos tres *commits* con nombres distintos. Al ejecutar todo el cuaderno, la última celda escribe `Dataset sintético eliminado (no debe subirse al repositorio).`: es la limpieza de las fotos generadas, y es normal.

Hecho cuando: otra persona podría ejecutar el cuaderno siguiendo solo el README, la sección «Revisión con la rúbrica» está rellena y los tres *commits* están en el repositorio.

Después, 30 minutos de entrega y review: cada persona rellena y sube su autoevaluación y coevaluación a `sprint-11/`, y el equipo enseña, en unos 5 minutos, el demostrador funcionando con 2 o 3 fotos, la matriz de confusión y tres errores comentados («el modelo confunde X con Y porque… y propondríamos…»). Después, la retro.

Si algo falla

Veis…	Qué significa	Qué hacer
El archivo ha caído en la raíz del repositorio y no en `sprint-11/`	Se subió sin entrar antes en la carpeta.	Abrid el archivo → icono del lápiz → en la ruta de arriba escribid `sprint-11/` delante del nombre → **Commit changes**. Lo mueve.
El archivo no aparece tras subirlo	Falta pulsar el botón verde **Commit changes**: sin él no se guarda nada.	Volved a subirlo y pulsadlo.
No veis el botón *Add file*	No sois colaboradores del repositorio o no habéis aceptado la invitación.	Pedid a quien creó el repositorio que os añada y aceptad la invitación.
El cuaderno subido no muestra gráficas ni cifras	Se descargó sin haberlo ejecutado.	*Entorno de ejecución → Reiniciar y ejecutar todo* y descargad de nuevo el `.ipynb`.
Reiniciar y ejecutar todo se para en una celda con `...`	Falta rellenar el hueco de un ejercicio.	Completad los ejercicios 1 a 5, en orden, y repetid.
El archivo se llama `Copia de S11_04…ipynb` o lleva espacios	Colab conserva el nombre de la copia.	Renombradlo a `reto_vision_cliente.ipynb` (minúsculas, sin espacios) antes de descargarlo.

Veis...	Qué significa	Qué hacer
Las cifras del cuaderno cambian un poco al ejecutar otra vez	El entrenamiento de una red no es idéntico al 100 % entre ejecuciones y ordenadores.	Usad en el informe las cifras de la ejecución final (la de *Reiniciar y ejecutar todo*).

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **suena muy segura aunque se equivoque** y no ha visto vuestras fotos. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea o por un error del informe, tenéis que saber explicarlo.
- **Las hipótesis se comprueban mirando la foto.** La IA propone causas típicas; vosotros dejáis solo las que **se ven en vuestras fotos**.
- **Declaradlo** en el README: qué pedisteis, qué revisasteis y qué cambiasteis.
- **Nada de datos personales ni claves** en una IA, y **no subáis fotos con personas** a ningún servicio.

Para qué	Ejemplo de petición	Qué comprobáis
Entender el *transfer learning*	«Explícame transfer learning con la analogía de un cocinero experto que aprende un plato nuevo»	Que podéis contarlo con vuestras palabras
Interpretar una matriz	«Mi modelo confunde mucho camping con casa rural. ¿Qué causas habituales puede haber?»	Que las contrastáis mirando las **fotos reales**
Entender un error de Keras	«Me sale este error al cargar las imágenes: *[mensaje completo]*. Explícame qué significa, sin darme todavía la solución»	Que entendéis la causa y lo arregláis vosotros
Revisar el informe	«Léelo como si fueras el cliente y dime qué no se entiende»	Que no cambia ninguna cifra

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
`AssertionError` al ejecutar un ejercicio	La función no devuelve lo que pide el enunciado	Leer el mensaje del `assert`: dice qué esperaba. Probar con un ejemplo pequeño
La base original queda congelada	Se pone `trainable = False` sobre la base y no sobre la copia	Trabajar con la copia (`clone_model`); el ejercicio 2 lo comprueba
Fotos casi idénticas en entrenamiento y prueba	Se parte sin cuidado o se generan variantes antes de partir	No tocar la partición del cuaderno; si hacéis aumento de datos, solo con las fotos de entrenamiento
Pocas fotos en alguna clase	Es difícil conseguirlas	Mínimo 30 por clase; si no llegáis, avisad del límite en el informe
Fiarse solo de la exactitud global	Es un solo número y esconde las clases que fallan	Mirar las **métricas por clase** y la matriz de confusión
Analizar errores sin mirar las fotos	Es más cómodo	Abrir las fotos mal clasificadas una a una
Usar la prueba para decidir	Se prueba «a ver qué sale» varias veces	Decidir con validación; la prueba se mira **una vez**
Umbral de confianza «a ojo»	Se pone 0,5 sin más	Medir cobertura y acierto de varios umbrales
Usar fotos de internet sin licencia	«Total, es un curso»	Solo fotos propias o con licencia abierta; guardar autor y licencia

Ayuda 3 · Si os atascáis

1. **Colab dice que no hay memoria o se desconecta:** reducid el tamaño de las imágenes o el número de fotos, y usad *Entorno de ejecución → Reiniciar y ejecutar todo*.
2. **El modelo acierta siempre la misma clase:** hay desequilibrio; comprobad con `contar_por_clase` cuántas fotos hay en cada clase.
3. **No sabemos explicar un error:** mirad la foto como si fuerais el modelo: ¿qué rasgo la haría parecerse a la otra clase?
4. **Hay menos de 6 errores en la prueba:** analizadlos todos y completad hasta 6 con errores de validación, diciéndolo en la tabla. Preguntad al docente si dudáis.
5. Repasad el apartado del paso en el que estáis y el código explicado del cuaderno; después, preguntad al equipo y, por último, al docente.